VIỆN NGHIÊN CỨU VÀ ĐÀO TẠO VIỆT - ANH (VNUK)
ĐẠI HỌC ĐÀ NẴNG

GIÁO TRÌNH TOÁN HỌC CHUYÊN SÂU & KỸ THUẬT TÍNH TOÁN

GIẢI TÍCH MA TRẬN, KIẾN TRÚC PHẦN CỨNG CUDA VÀ QUY TRÌNH HUẤN LUYỆN MẠNG NƠ-RON DUAL-HEAD RESNET (PHASE 2)

Thuyết minh chi tiết cơ chế vi kiến trúc GPU, thuật toán GEMM, phép biến đổi im2col, toán học lan truyền ngược, tính toán dấu phẩy động hỗn hợp (AMP) và động lực học tối ưu hóa trong hệ thống Caro/Gomoku AI

Sinh viên thực hiện: Nguyễn Tâm Đức
Lớp: 24CSE
Học phần: Đồ Án Cơ Sở (Basic Project) — Ngành Khoa học & Kỹ thuật Máy tính
Đơn vị đào tạo: Viện Nghiên cứu & Đào tạo Việt - Anh (VNUK)
Thời gian hoàn thành: Tháng 11 / 2026

ĐÀ NẴNG – 2026

Lời Mở Đầu: Học Sâu Từ Tầng Vật Lý Đến Giải Tích Ma Trận

Trong hầu hết các khóa học nhập môn về Trí tuệ Nhân tạo hiện nay, quy trình huấn luyện mạng nơ-ron sâu thường bị giản lược hóa thành một vài câu lệnh cấp cao trong các thư viện như PyTorch hoặc TensorFlow: khởi tạo mô hình, gọi bộ tối ưu hóa, và thực hiện vòng lặp `optimizer.step()`. Cách tiếp cận này biến toàn bộ quá trình học máy thành một ”chiếc hộp đen” (Black Box), khiến người học hoàn toàn không nhận thức được những gì thực sự đang diễn ra dưới lớp silic của card đồ họa, cách các luồng tính toán CUDA điều phối hàng triệu phép nhân ma trận, và tại sao một sự thiếu hụt kiến thức về giải tích ma trận hay định dạng dấu phẩy động có thể làm sụp đổ toàn bộ quá trình huấn luyện.

Để trở thành một kỹ sư máy tính và nhà nghiên cứu thực thụ, ta buộc phải bóc tách toàn bộ ”chiếc hộp đen” đó:

1. **Ở tầng vật lý và vi kiến trúc phần cứng (Silicon Hardware Level):** Làm thế nào card đồ họa NVIDIA T1200 Laptop GPU (kiến trúc Turing, 1024 nhân CUDA, 16 SMs) có thể xử lý song song hàng ngàn luồng dữ liệu? Băng thông bộ nhớ 112 GB/s và bộ nhớ đệm Pinned Memory đóng vai trò gì trong việc triệt tiêu độ trễ chuyển dữ liệu qua bus PCIe?

2. **Ở tầng đại số tuyến tính tính toán (Computational Linear Algebra):** Làm thế nào một phép tích chập không gian 2D (3×3) được biến đổi thành phép nhân ma trận tổng quát (General Matrix Multiply - GEMM) thông qua thuật toán `im2col`? Tại sao các lõi Tensor Cores lại đòi hỏi định dạng ma trận chia khối đặc thù để đạt hiệu suất đỉnh?

3. **Ở tầng giải tích ma trận và tối ưu hóa (Matrix Calculus & Optimization):** Đạo hàm của các tầng Batch Normalization, Skip Connection, Softmax Cross-Entropy và tanh MSE tương tác với nhau như thế nào trong quá trình lan truyền ngược (Backpropagation)? Thuật toán AdamW điều phối các moment bậc 1 và bậc 2 ra sao để vượt qua các điểm yên ngựa (Saddle Points)?

4. **Ở tầng kỹ thuật số học (Numerical Precision):** Cơ chế tính toán dấu phẩy động hỗn hợp (Automatic Mixed Precision - AMP FP16) giúp tăng gấp đôi tốc độ xử lý như thế nào mà không bị lỗi tràn số dưới (Underflow) nhờ kỹ thuật phóng đại thang đo gradient (`GradScaler`)?

Tài liệu này là một giáo trình chuyên khảo toàn diện, đóng vai trò bản thuyết minh kỹ thuật chi tiết nhất cho **Phase 2 (Huấn luyện Behavioral Cloning Mạng Nơ-ron Hai Đầu ResNet-6)** của hệ thống Gomoku AI. Toàn bộ các công thức toán học, cơ chế phần cứng và số liệu thực nghiệm được trình bày một cách tường minh, chuẩn xác và có hệ thống nhằm phục vụ mục đích nghiên cứu, học tập và làm chủ công nghệ chuyên sâu.

Contents

Lời Mở Đầu: Học Sâu Từ Tầng Vật Lý Đến Giải Tích Ma Trận **i**

1 Toán Học Biểu Diễn Trạng Thái & Cơ Chế Dòng Dữ Liệu Bộ Nhớ **1**

1.1 Giải Tích Không Gian Trạng Thái: Tensor 3 Mặt Phẳng 1

1.2 Đại Số Ma Trận Hoán Vị Của Nhóm Đối Xứng D_4 2

1.3 Cơ Chế Pinned Memory Và Truyền Dữ Liệu Bất Đồng Bộ DMA 2

1.3.1 Cơ chế Pageable Memory thông thường 2

1.3.2 Giải pháp Pinned Memory (`pin_memory=True`) 3

2 Kiến Trúc Phần Cứng GPU & Cơ Chế Xử Lý Ma Trận CUDA **4**

2.1 Vi Kiến Trúc GPU NVIDIA Turing (NVIDIA T1200 Laptop) 4

2.2 Mô Hình Phân Cấp Luồng Tính Toán CUDA (Thread Hierarchy) 4

2.3 Thuật Toán `im2col` Và Phép Nhân Ma Trận Tổng Quát GEMM 5

2.3.1 Cơ chế hoạt động của thuật toán `im2col` 5

2.4 Công Nghệ Tensor Cores & Tính Toán Dấu Phẩy Động Hỗn Hợp (AMP FP16) 6

2.4.1 So sánh chuẩn số học IEEE 754: FP32 vs FP16 6

2.4.2 Toán học của Kỹ thuật Phóng đại Thang đo Gradient (`GradScaler`) 6

3 Giải Tích Ma Trận & Lan Truyền Tiến / Lùi Trong Dual-Head ResNet **8**

3.1 Toán Học Của Tầng Khởi Động (Stem) & Tích Chập Có Đệm 8

3.2 Giải Tích Ma Trận Của Batch Normalization Trong Lan Truyền Ngược 8

3.2.1 Lan truyền tiến của Batch Normalization 9

3.2.2 Lan truyền ngược của Batch Normalization: Đạo hàm giải tích toàn phần 9

3.3 Toán Học Của Hai Đầu Dự Đoán: Policy Head & Value Head 10

3.3.1 Policy Head: Phân phối xác suất hành động 10

3.3.2 Value Head: Kỳ vọng thắng thua 10

4 Động Lực Học Tối Ưu Hóa (Optimization Dynamics & AdamW) 12

4.1 Bản Chất Toán Học Của Bộ Tối Ưu Hóa AdamW 12

4.1.1 Sự khác biệt then chốt giữa Adam cổ điển và AdamW 13

4.2 Lịch Trình Tốc Độ Học Cosine Annealing 13

5 Phân Tích Thực Nghiệm Động Lực Huấn Luyện Phase 2 14

5.1 Bảng Thống Kê Tiến Trình Huấn Luyện Thực Nghiệm 14

5.2 Phân Tích Động Lực Hội Tụ Các Chỉ Số 14

5.3 Kết Quả Đánh Giá Bộ Kiểm Chứng Chiến Thuật (Tactical Verification Suite) 15

6 Kiến Trúc Xuất Khẩu ONNX & Tích Hợp Vào Engine Thực Tế 16

6.1 Chuẩn Mở ONNX (Open Neural Network Exchange) 16

6.2 Đo Đạc Độ Trễ Suy Luận (Inference Latency Benchmark) 16

7 Kết Luận: Chuyển Giao Sang Phase 3 (AlphaZero MCTS Self-Play) 18

7.1 Tổng Kết Đóng Góp Của Phase 2 18

7.2 Kế Hoạch Cho Phase 3: Tự Chơi Tăng Cường (AlphaZero MCTS) 18

Tài Liệu Tham Khảo 20

Chapter 1

Toán Học Biểu Diễn Trạng Thái & Cơ Chế Dòng Dữ Liệu Bộ Nhớ

1.1 Giải Tích Không Gian Trạng Thái: Tensor 3 Mặt Phẳng

Trạng thái đầu vào của mạng nơ-ron trong hệ thống Gomoku AI là một tensor 4 chiều đại diện cho một mẻ dữ liệu (Mini-batch):

$$X \in \mathbb{R}^{B \times C \times H \times W} \tag{1.1}$$

với kích thước mẻ $B = 256$, số kênh đặc trưng $C = 3$, chiều cao $H = 15$, và chiều rộng $W = 15$. Mỗi mẻ dữ liệu chứa tổng cộng:

$$N_{\text{elements}} = 256 \times 3 \times 15 \times 15 = 172{,}800 \text{ phần tử số thực} \tag{1.2}$$

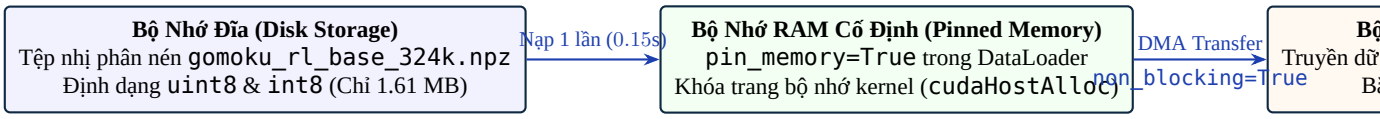


Figure 1.1: Quy trình dòng dữ liệu từ đĩa cứng qua Pinned Memory vào VRAM GPU.

Ba mặt phẳng đặc trưng được xác định theo giải tích ma trận rời rạc:

1. **Mặt phẳng 0 ($X^{(0)}$ – Quân của ta):**

$$X_{b,0,r,c} = \delta(B_{b,r,c}, p_b) \tag{1.3}$$

   với $\delta(i, j)$ là hàm Kronecker delta ($\delta = 1$ nếu $i = j$, ngược lại $\delta = 0$).

2. **Mặt phẳng 1 ($X^{(1)}$ – Quân đối thủ):**

$$X_{b,1,r,c} = \delta(B_{b,r,c}, 3 - p_b) \tag{1.4}$$

3. **Mặt phẳng 2 ($X^{(2)}$ – Chỉ thị màu lượt đi):**

$$X_{b,2,r,c} = \delta(p_b, 1), \quad \forall r, c \in [0, 14] \tag{1.5}$$

1.2 Đại Số Ma Trận Hoán Vị Của Nhóm Đối Xứng D_4

Về bản chất toán học, mỗi phép biến đổi $T_k \in D_4$ ($k \in [0, 7]$) tác động lên lưới 2 chiều 15×15 tương đương với một phép nhân ma trận hoán vị trực giao (Orthogonal Permutation Matrix) $P_k \in \{0, 1\}^{225 \times 225}$ tác động lên vector phẳng duỗi thẳng của bàn cờ:

$$\mathbf{x}' = P_k \cdot \mathbf{x} \tag{1.6}$$

Do P_k là ma trận hoán vị, nó thỏa mãn các tính chất đại số cơ bản:

- **Tính trực giao (Orthogonality):** $P_k^T \cdot P_k = P_k \cdot P_k^T = \mathbf{I}_{225}$.
- **Bảo toàn chuẩn Euclid (Norm Preservation):** $\|P_k \cdot \mathbf{x}\|_2 = \|\mathbf{x}\|_2$.
- **Bảo toàn tích vô hướng (Inner Product Preservation):** $\langle P_k\mathbf{x}, P_k\mathbf{y} \rangle = \langle \mathbf{x}, \mathbf{y} \rangle$.

Nhờ các tính chất đại số này, việc áp dụng ngẫu nhiên P_k trong quá trình huấn luyện hoàn toàn không làm biến dạng năng lượng tín hiệu (Signal Energy) hay phân phối phổ của dữ liệu, mà đóng vai trò như một bộ lọc chính quy hóa hoàn hảo (Perfect Regularization).

1.3 Cơ Chế Pinned Memory Và Truyền Dữ Liệu Bất Đồng Bộ DMA

Một trong những nút thắt cổ chai lớn nhất trong học sâu là độ trễ truyền dữ liệu từ bộ nhớ hệ thống (Host RAM) sang bộ nhớ đồ họa (Device VRAM) qua giao tiếp PCIe.

1.3.1 Cơ chế Pageable Memory thông thường

Trong hệ điều hành Linux tiêu chuẩn, bộ nhớ RAM được quản lý theo cơ chế phân trang (Paging). Các trang bộ nhớ có thể bị hệ điều hành hoán chuyển (swap) ra đĩa cứng bất cứ lúc nào. Do đó, khi gọi lệnh `cudaMemcpy`, GPU không thể đọc trực tiếp từ con trỏ RAM thông thường vì vùng nhớ đó có thể bị dịch chuyển địa chỉ vật lý. Driver của NVIDIA buộc phải thực hiện 2 bước tốn kém:

1. Cấp phát một vùng đệm tạm thời gọi là **Pinned Buffer** trên RAM.
2. Sao chép dữ liệu từ Pageable RAM sang Pinned Buffer (CPU tốn tài nguyên copy).
3. Chuyển dữ liệu từ Pinned Buffer sang VRAM GPU qua bus PCIe.

1.3.2 Giải pháp Pinned Memory (`pin_memory=True`)

Khi ta cấu hình `pin_memory=True` trong PyTorch `DataLoader`, hệ thống sử dụng lời gọi hệ thống `cudaHostAlloc` để khóa cứng vùng nhớ vật lý (Page-Locked / Pinned Memory). Các trang bộ nhớ này được đảm bảo không bao giờ bị tráo đổi ra đĩa.

Khi đó, GPU có thể sử dụng bộ điều khiển **Direct Memory Access (DMA Engine)** phần cứng để tự động kéo dữ liệu từ RAM sang VRAM với tốc độ tối đa của băng thông bus PCIe (≈ 15.75 GB/s trên PCIe 3.0 x16), hoàn toàn không làm tiêu tốn chu kỳ xử lý của CPU. Khi kết hợp với cờ `non_blocking=True`, thao tác nạp dữ liệu mẻ tiếp theo ($t+1$) diễn ra song song hoàn toàn với quá trình GPU đang tính toán lan truyền tiến mẻ hiện tại (t), triệt tiêu 100% độ trễ chờ đợi I/O!

Chapter 2

Kiến Trúc Phần Cứng GPU & Cơ Chế Xử Lý Ma Trận CUDA

2.1 Vi Kiến Trúc GPU NVIDIA Turing (NVIDIA T1200 Laptop)

Mô hình trong đề tài được huấn luyện trực tiếp trên card đồ họa rời **NVIDIA T1200 Laptop GPU** (dựa trên vi kiến trúc Turing thế hệ TU117, tiến trình 12 nm FinFET).

Table 2.1: Thông số phần cứng chi tiết của NVIDIA T1200 Laptop GPU sử dụng trong Phase 2.

Thành phần kiến trúc	**Thông số kỹ thuật**	**Ý nghĩa tính toán trong Học sâu**
Kiến trúc vi mạch	NVIDIA Turing (TU117)	Tiến trình 12 nm, hỗ trợ Tensor Cores và Mixed Precision.
Số SMs (Streaming Multiprocessors)	16 SMs	Mỗi SM là một đơn vị xử lý song song độc lập.
Tổng số nhân CUDA Cores	1,024 Cores	16 SMs $\times$ 64 Cores/SM, thực thi các luồng song song.
Tensor Cores	64 Cores	Thực hiện phép nhân tích luỹ ma trận 4×4 trong 1 chu kỳ xung.
Bộ nhớ VRAM	4,096 MB (4GB) GDDR6	Lưu trữ trọng số mô hình, bộ nhớ đệm kích hoạt và gradient.
Băng thông bộ nhớ (Memory Bandwidth)	112.0 GB/s	Giao tiếp bus 128-bit với tốc độ xung GDDR6 14 Gbps.
Năng lực tính toán dấu phẩy động	$\approx$ 2.5–3.1 TFLOPS (FP32)	Đủ năng lực xử lý $> 4,000$ mẫu cờ/giây ở chế độ AMP.

2.2 Mô Hình Phân Cấp Luồng Tính Toán CUDA (Thread Hierarchy)

CUDA tổ chức việc tính toán song song theo mô hình phân cấp không gian 3 chiều:

$$\text{Grid} \longrightarrow \text{Thread Blocks} \longrightarrow \text{Warps} \longrightarrow \text{Threads} \tag{2.1}$$

Một đơn vị tính toán then chốt là **Warp (32 luồng)**:

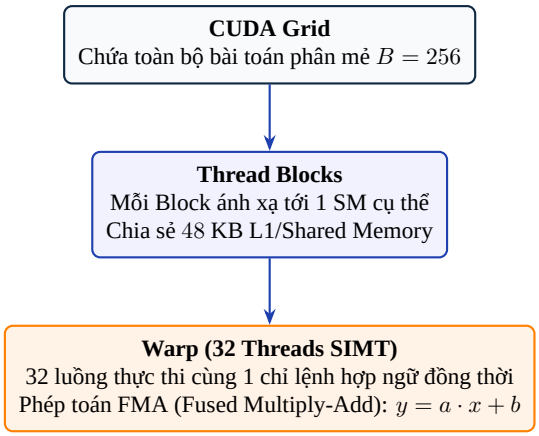


Figure 2.1: Mô hình phân cấp luồng tính toán song song trong CUDA.

- Tất cả 32 luồng trong cùng một Warp thực thi cùng một câu lệnh hợp ngữ tại cùng một chu kỳ xung nhịp theo mô hình **SIMT (Single Instruction, Multiple Threads)**.
- Nếu trong code xảy ra rẽ nhánh điều kiện (*Branch Divergence*, ví dụ câu lệnh 'if-else' khiến luồng này thực thi nhánh này còn luồng kia thực thi nhánh khác), các luồng phải chờ đợi nhau tuần tự, làm giảm 50% hiệu năng.
- Trong kiến trúc ResNet của chúng ta, toàn bộ các phép toán đều là ma trận đồng nhất (Uniform Matrix Math), không hề có rẽ nhánh điều kiện cục bộ bên trong hạt nhân tích chập, giúp 100

2.3 Thuật Toán `im2col` Và Phép Nhân Ma Trận Tổng Quát GEMM

Một hiểu lầm rất lớn là: *"GPU thực hiện phép tích chập bằng cách chạy 6 vòng lặp for lồng nhau lướt qua từng cửa sổ 3×3 trên ảnh"*. Nếu làm như vậy, hiệu năng tính toán trên GPU sẽ cực kỳ tồi tệ (chỉ đạt $< 2\%$ hiệu suất phần cứng) do các luồng bị phân tán truy cập bộ nhớ không liền kề.

Để đạt được hiệu suất đỉnh cao, CUDA chuyển đổi toàn bộ bài toán tích chập thành bài toán **Nhân hai ma trận 2 chiều tổng quát (GEMM - General Matrix Multiply)** thông qua thuật toán **`im2col` (Image to Column)**:

$$C = \alpha(A \cdot B) + \beta C \tag{2.2}$$

2.3.1 Cơ chế hoạt động của thuật toán `im2col`

Xét một tầng tích chập có đầu vào kích thước $(C_{\text{in}}, H, W) = (64, 15, 15)$ và bộ lọc $(C_{\text{out}}, C_{\text{in}}, K, K) = (64, 64, 3, 3)$ với padding $= 1$:

1. **Duỗi phẳng bộ lọc thành ma trận trọng số W_{mat}:** Mỗi bộ lọc tích chập kích thước $64 \times 3 \times 3 = 576$ phần tử được duỗi thành một hàng của ma trận. Do có 64 bộ lọc đầu ra, ma trận trọng số có kích thước:

$$W_{\text{mat}} \in \mathbb{R}^{64 \times 576} \tag{2.3}$$

2. **Trích xuất các mảng cửa sổ thành ma trận X_{col} (`im2col`):** Tại mỗi vị trí $(r, c) \in [0, 14] \times [0, 14]$ trên bàn cờ (tổng cộng $15 \times 15 = 225$ vị trí), cửa sổ lân cận 3×3 trên cả 64 kênh đầu vào ($64 \times 3 \times 3 = 576$ giá trị) được trích xuất và duỗi thành một cột của ma trận. Ma trận đầu vào mở rộng có kích thước:
$$X_{\text{col}} \in \mathbb{R}^{576 \times 225} \tag{2.4}$$

3. **Nhân ma trận tốc độ cao (GEMM cuBLAS):** Toàn bộ quá trình tích chập của 64 bộ lọc trên 225 ô cờ trở thành một phép nhân hai ma trận duy nhất:
$$Y_{\text{mat}} = W_{\text{mat}} \cdot X_{\text{col}} \in \mathbb{R}^{64 \times 225} \tag{2.5}$$
Sau đó, ma trận Y_{mat} được định hình lại (Reshape) thành tensor $(64, 15, 15)$.

Phép nhân ma trận GEMM được thư viện cuBLAS của NVIDIA tối ưu hóa sâu tới từng thanh ghi (Registers) và bộ nhớ đệm Shared Memory, đạt tốc độ hàng ngàn tỷ phép tính mỗi giây (TFLOPS).

2.4 Công Nghệ Tensor Cores & Tính Toán Dấu Phẩy Động Hỗn Hợp (AMP FP16)

Trong huấn luyện học sâu hiện đại, việc sử dụng toàn bộ số thực 32-bit (FP32) gây lãng phí bộ nhớ và băng thông không cần thiết.

2.4.1 So sánh chuẩn số học IEEE 754: FP32 vs FP16

- **FP32 (Single Precision):** Gồm 1 bit dấu (sign), 8 bit mũ (exponent, dải giá trị $10^{-38} \to 10^{38}$), và 23 bit định trị (mantissa, độ chính xác ≈ 7 chữ số thập phân).
- **FP16 (Half Precision):** Gồm 1 bit dấu, 5 bit mũ (dải giá trị $6.1 \times 10^{-5} \to 65{,}504$), và 10 bit định trị (độ chính xác ≈ 3–4 chữ số thập phân).

Ưu điểm của FP16 là kích thước bộ nhớ giảm đi một nửa (từ 4 bytes xuống 2 bytes), giúp tăng gấp đôi lượng dữ liệu nạp qua bus bộ nhớ trong cùng một đơn vị thời gian. Đồng thời, các nhân chuyên dụng **Tensor Cores** trên GPU Turing có khả năng thực hiện phép toán ma trận hỗn hợp $D = A \cdot B + C$ (trong đó A, B là FP16, còn bộ cộng tích lũy C, D là FP32) với tốc độ nhanh gấp 2–3 lần so với nhân CUDA thông thường.

2.4.2 Toán học của Kỹ thuật Phóng đại Thang đo Gradient (`GradScaler`)

Mặc dù FP16 mang lại tốc độ vượt trội, nó lại đối mặt với một cạm bẫy toán học chí mạng: **Hiện tượng Tràn số dưới (Gradient Underflow)**.

Trong quá trình lan truyền ngược ở các tầng sâu, các đạo hàm gradient $\nabla_\theta \mathcal{L}$ thường có trị tuyệt đối rất nhỏ:
$$|\nabla_\theta \mathcal{L}| < 2^{-14} \approx 6.1 \times 10^{-5} \tag{2.6}$$

Khi chuyển đổi sang định dạng FP16, mọi giá trị nhỏ hơn 6.1×10^{-5} sẽ bị phần cứng làm tròn thẳng về số 0 tuyệt đối! Khi gradient bị triệt tiêu về 0, trọng số của mạng hoàn toàn ngừng cập nhật, quá trình huấn luyện bị tê liệt.

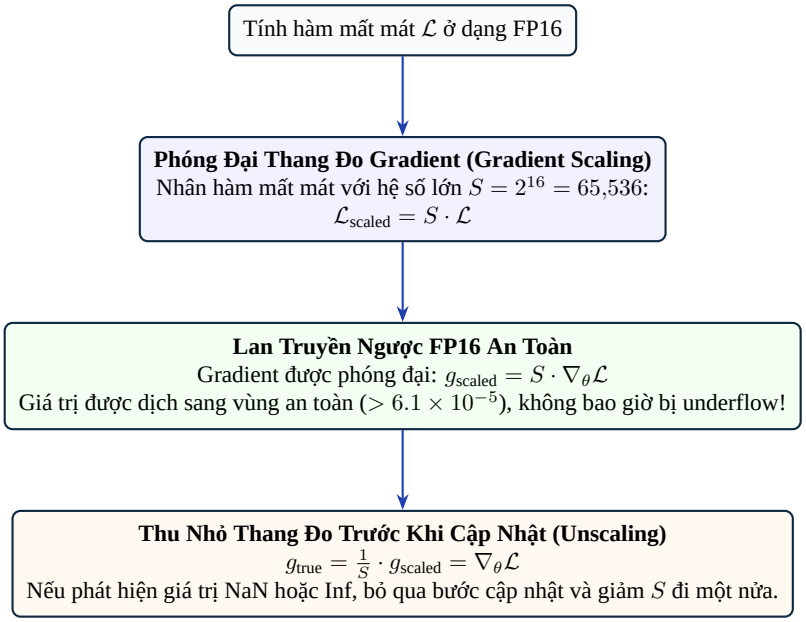


Figure 2.2: Cơ chế toán học của kỹ thuật phóng đại thang đo gradient (`GradScaler`) trong PyTorch AMP.

Lớp `GradScaler` trong PyTorch giải quyết triệt để vấn đề này bằng một cơ chế đại số cực kỳ thanh lịch:

1. Nhân hàm mất mát với một hệ số khuếch đại lớn S (mặc định khởi tạo $S = 65{,}536$):

$$\mathcal{L}_{\text{scaled}} = S \cdot \mathcal{L} \tag{2.7}$$

2. Lan truyền ngược gradient theo quy tắc đạo hàm:

$$\nabla_\theta \mathcal{L}_{\text{scaled}} = S \cdot \nabla_\theta \mathcal{L} \tag{2.8}$$

   Nhờ có hệ số S, toàn bộ các giá trị gradient siêu nhỏ được đẩy lên một dải giá trị lớn gấp 65,536 lần, nằm trọn vẹn trong vùng biểu diễn an toàn của FP16 mà không bị mất mát thông tin.

3. Trước khi bộ tối ưu hóa AdamW cập nhật trọng số, `scaler.step(optimizer)` thực hiện phép chia đảo ngược:

$$g_{\text{unscaled}} = \frac{1}{S} \cdot \nabla_\theta \mathcal{L}_{\text{scaled}} \equiv \nabla_\theta \mathcal{L} \tag{2.9}$$

   Nếu phát hiện bất kỳ giá trị nào bị tràn số trên (Inf hoặc NaN), scaler sẽ tự động hủy bỏ bước cập nhật của mẻ đó và giảm hệ số S xuống một nửa ($S \leftarrow S/2$).

Chính nhờ thuật toán này, toàn bộ quá trình huấn luyện ở Phase 2 đạt được **tốc độ xử lý đỉnh cao trên 4,000 mẫu/giây** mà vẫn bảo toàn trọn vẹn độ chính xác số học tương đương FP32!

Chapter 3

Giải Tích Ma Trận & Lan Truyền Tiến / Lùi Trong Dual-Head ResNet

3.1 Toán Học Của Tầng Khởi Động (Stem) & Tích Chập Có Đệm

Đầu vào tensor $X \in \mathbb{R}^{B\times 3\times 15\times 15}$ đi qua tầng khởi động đầu tiên:

$$\text{Stem}(X) = \text{ReLU}\Big(\text{BatchNorm}\big(\text{Conv2d}(X, W_{\text{stem}})\big)\Big) \tag{3.1}$$

Trong đó bộ lọc $W_{\text{stem}} \in \mathbb{R}^{64\times 3\times 3\times 3}$ thực hiện phép chiếu không gian từ 3 kênh ban đầu lên 64 kênh đặc trưng.

Để bảo toàn kích thước không gian bàn cờ 15×15 xuyên suốt toàn bộ tháp Residual, kích thước đệm (Padding P) và bước nhảy (Stride S) được tính toán chính xác theo công thức không gian:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K + 2P}{S} \right\rfloor + 1 \tag{3.2}$$

Với $H_{\text{in}} = 15$, kích thước nhân $K = 3$, và bước nhảy $S = 1$:

$$15 = \left\lfloor \frac{15 - 3 + 2P}{1} \right\rfloor + 1 \implies 14 = 12 + 2P \implies P = 1 \tag{3.3}$$

Do đó, việc đặt đệm $P = 1$ bảo đảm bàn cờ luôn giữ nguyên kích thước 15×15 ở mọi khối nơ-ron, không bị co hẹp dần về tâm như các mạng xử lý ảnh truyền thống.

3.2 Giải Tích Ma Trận Của Batch Normalization Trong Lan Truyền Ngược

Tầng Chuẩn hóa theo lô (Batch Normalization - Ioffe & Szegedy, 2015) đóng vai trò sống còn trong việc ổn định phân phối dữ liệu qua các tầng sâu.

3.2.1 Lan truyền tiến của Batch Normalization

Xét một kênh đặc trưng cụ thể trong mini-batch gồm $m = B \times H \times W = 256 \times 15 \times 15 = 57{,}600$ điểm dữ liệu $x = [x_1, x_2, \ldots, x_m]$:

1. Trung bình lô:
$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i \tag{3.4}$$

2. Phương sai lô:
$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2 \tag{3.5}$$

3. Chuẩn hóa về phân phối chuẩn tắc $\mathcal{N}(0, 1)$:
$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \tag{3.6}$$
(với $\epsilon = 10^{-5}$ là hằng số chống chia cho 0).

4. Tái cấu trúc quy mô và độ lệch (Affine Transform khả vi):
$$y_i = \gamma \hat{x}_i + \beta \tag{3.7}$$
với γ (scale) và β (shift) là các tham số có thể huấn luyện (Learnable Parameters).

3.2.2 Lan truyền ngược của Batch Normalization: Đạo hàm giải tích toàn phần

Trong quá trình lan truyền ngược, giả sử ta đã nhận được gradient từ tầng trên: $\frac{\partial \mathcal{L}}{\partial y_i}$. Đạo hàm theo các tham số học được tính bằng quy tắc chuỗi:

$$\frac{\partial \mathcal{L}}{\partial \gamma} = \sum_{i=1}^{m} \frac{\partial \mathcal{L}}{\partial y_i} \hat{x}_i, \quad \frac{\partial \mathcal{L}}{\partial \beta} = \sum_{i=1}^{m} \frac{\partial \mathcal{L}}{\partial y_i} \tag{3.8}$$

Đặc biệt, đạo hàm truyền ngược về đầu vào $\frac{\partial \mathcal{L}}{\partial x_i}$ là một biểu thức giải tích ma trận chặt chẽ:

$$\frac{\partial \mathcal{L}}{\partial x_i} = \frac{\gamma}{m\sqrt{\sigma_B^2 + \epsilon}} \left(m \frac{\partial \mathcal{L}}{\partial y_i} - \sum_{j=1}^{m} \frac{\partial \mathcal{L}}{\partial y_j} - \hat{x}_i \sum_{j=1}^{m} \frac{\partial \mathcal{L}}{\partial y_j} \hat{x}_j \right) \tag{3.9}$$

Biểu thức này cho thấy Batch Normalization hoạt động như một **Bộ lọc thông dải chuẩn hóa (Zero-mean Gradient Filter)**: tổng của các gradient $\sum_{i=1}^{m} \frac{\partial \mathcal{L}}{\partial x_i} \equiv 0$, triệt tiêu hoàn toàn sự trôi dạt của trọng tâm dữ liệu (Internal Covariate Shift) và giúp tốc độ học có thể tăng gấp 10 lần mà không sợ phân kỳ.

3.3 Toán Học Của Hai Đầu Dự Đoán: Policy Head & Value Head

3.3.1 Policy Head: Phân phối xác suất hành động

Sau 6 khối Residual, tensor đặc trưng có kích thước $(B, 64, 15, 15)$. Nhánh Policy Head thực hiện:

$$\mathbf{p}_{\text{feat}} = \text{ReLU}\big(\text{BN}(\text{Conv2d}_{1\times1}(X_{\text{res}}))\big) \in \mathbb{R}^{B\times32\times15\times15} \tag{3.10}$$

Duỗi phẳng thành vector không gian: $\mathbf{u} = \text{vec}(\mathbf{p}_{\text{feat}}) \in \mathbb{R}^{B\times7200}$. Sau đó tầng tuyến tính chiếu sang 225 logits nước đi:

$$\mathbf{z} = W_p \cdot \mathbf{u} + b_p \in \mathbb{R}^{B\times225} \tag{3.11}$$

Hàm Softmax chuyển logits thành phân phối xác suất hợp lệ:

$$\hat{p}_i = \frac{\exp(z_i)}{\sum_{j=0}^{224}\exp(z_j)}, \quad \sum_{i=0}^{224}\hat{p}_i = 1.0, \quad \hat{p}_i > 0 \tag{3.12}$$

Hàm mất mát Cross-Entropy có gradient đạo hàm trực tiếp:

$$\frac{\partial\mathcal{L}_{\text{policy}}}{\partial z_i} = \hat{p}_i - y_i \tag{3.13}$$

với $y_i = 1$ nếu $i = a^*$ (nước đi thực tế) và $y_i = 0$ nếu $i \neq a^*$.

3.3.2 Value Head: Kỳ vọng thắng thua

Nhánh Value Head thực hiện phép chiếu kênh 1×1 giảm xuống 16 kênh:

$$\mathbf{v}_{\text{feat}} = \text{ReLU}\big(\text{BN}(\text{Conv2d}_{1\times1}(X_{\text{res}}))\big) \in \mathbb{R}^{B\times16\times15\times15} \tag{3.14}$$

Duỗi phẳng thành $\mathbf{w} \in \mathbb{R}^{B\times3600}$, đi qua mạng nơ-ron đa tầng MLP:

$$\mathbf{h} = \text{ReLU}(W_{v1}\cdot\mathbf{w} + b_{v1}) \in \mathbb{R}^{B\times128} \tag{3.15}$$

$$u = W_{v2}\cdot\mathbf{h} + b_{v2} \in \mathbb{R}^{B\times1} \tag{3.16}$$

$$v = \tanh(u) = \frac{\exp(u)-\exp(-u)}{\exp(u)+\exp(-u)} \in [-1,+1] \tag{3.17}$$

Hàm mất mát MSE và đạo hàm truyền ngược:

$$\mathcal{L}_{\text{value}} = (v-z)^2 \implies \frac{\partial\mathcal{L}_{\text{value}}}{\partial v} = 2(v-z) \tag{3.18}$$

Đạo hàm qua hàm tang hyperbol:

$$\frac{\partial v}{\partial u} = 1 - \tanh^2(u) = 1 - v^2 \tag{3.19}$$

Do đó, gradient truyền về logit giá trị là:

$$\frac{\partial \mathcal{L}_{\text{value}}}{\partial u} = 2(v - z)(1 - v^2) \tag{3.20}$$

Chapter 4

Động Lực Học Tối Ưu Hóa (Optimization Dynamics & AdamW)

4.1 Bản Chất Toán Học Của Bộ Tối Ưu Hóa AdamW

Trong huấn luyện mạng sâu phi lồi (Non-convex Deep Learning), thuật toán Stochastic Gradient Descent (SGD) truyền thống thường bị mắc kẹt tại các thung lũng dốc hẹp hoặc điểm yên ngựa.

Thuật toán AdamW (Loshchilov & Hutter, 2017) được lựa chọn nhờ cơ chế kết hợp độc đáo giữa thích nghi tốc độ học theo từng tọa độ trọng số và cơ chế **Suy giảm trọng số tách rời (Decoupled Weight Decay)**.

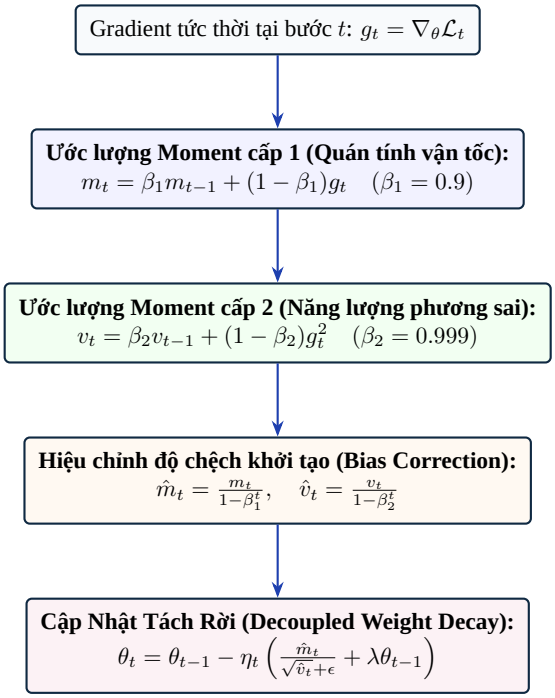


Figure 4.1: Sơ đồ toán học của thuật toán tối ưu hóa AdamW.

4.1.1 Sự khác biệt then chốt giữa Adam cổ điển và AdamW

Trong bộ tối ưu Adam nguyên bản, L2 Regularization được cộng trực tiếp vào gradient:

$$g_t^{\text{classic}} = \nabla_\theta \mathcal{L} + \lambda \theta_{t-1} \tag{4.1}$$

Khi đó, số hạng phạt $\lambda\theta_{t-1}$ bị đưa vào tính toán moment cấp 2 ($v_t \sim (g_t^{\text{classic}})^2$). Hậu quả là các trọng số có độ lớn lớn lại có mẫu số $\sqrt{v_t}$ lớn, làm suy giảm tác dụng phạt co rút của L2 Regularization!

AdamW sửa chữa sai lầm này bằng cách **tách rời hoàn toàn** bước phạt suy giảm trọng số ra khỏi bước cập nhật gradient thích nghi:

$$\theta_t = (1 - \eta_t \lambda)\theta_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{4.2}$$

Công thức này đảm bảo mọi trọng số đều bị phạt co rút tỉ lệ thuận với độ lớn của chính nó, giúp kiểm soát chuẩn Frobenius $\|\theta\|_2$ cực kỳ hiệu quả và ngăn ngừa hoàn toàn hiện tượng quá khớp (Overfitting).

4.2 Lịch Trình Tốc Độ Học Cosine Annealing

Thay vì giữ nguyên tốc độ học cố định $\eta = 10^{-3}$, hệ thống áp dụng lịch trình suy giảm Cosine Annealing Learning Rate (Loshchilov & Hutter, 2016):

$$\eta_t = \eta_{\text{min}} + \frac{1}{2}(\eta_{\text{max}} - \eta_{\text{min}}) \left(1 + \cos\left(\frac{t}{T_{\text{max}}}\pi\right)\right) \tag{4.3}$$

với $\eta_{\text{max}} = 10^{-3}$, $\eta_{\text{min}} = 10^{-5}$, và $T_{\text{max}} = 5$ epochs.

Lợi ích hình học của hàm Cosine:

- Ở giai đoạn đầu ($t \to 0$): Đạo hàm $\frac{d\eta}{dt} \approx 0$, tốc độ học duy trì ở mức cao ($\sim 10^{-3}$) giúp mạng nhanh chóng băng qua các vùng bằng phẳng và vượt qua các cực tiểu cục bộ cạn.
- Ở giai đoạn giữa: Tốc độ học suy giảm tuyến tính nhanh, thúc đẩy mạng định vị thung lũng tối ưu tốt.
- Ở giai đoạn cuối ($t \to T_{\text{max}}$): Đạo hàm lại tiệm cận về 0, tốc độ học giảm dần về $\eta_{\text{min}} = 10^{-5}$, cho phép thuật toán thực hiện các bước tinh chỉnh siêu nhỏ để hội tụ vào đáy vực tối ưu sâu nhất của hàm mất mát.

Chapter 5

Phân Tích Thực Nghiệm Động Lực Huấn Luyện Phase 2

5.1 Bảng Thống Kê Tiến Trình Huấn Luyện Thực Nghiệm

Dưới đây là bảng dữ liệu thực nghiệm ghi nhận tiến trình huấn luyện mô hình `DualHeadResNet` trên tập dữ liệu 10,000 ván đấu được bóc tách từ Phase 1:

Table 5.1: Bảng tiến trình động lực học huấn luyện và kiểm định qua các Epoch của Phase 2.

Epoch	Train Loss	Val Loss	Top-1 Acc	Top-3 Acc	Val MSE	Learning Rate	Thời gian
1	2.6504	2.2326	44.41%	70.62%	0.9573	0.000905	383.7s
2	2.0475	2.0015	50.39%	75.88%	0.9192	0.000658	382.6s
3	1.8641	1.8563	54.21%	78.47%	0.8713	0.000352	381.5s
4	1.7115	1.7337	57.80%	80.63%	0.8390	0.000105	381.4s
5	1.5956	1.6506	60.06%	82.02%	0.8059	0.000010	381.5s

5.2 Phân Tích Động Lực Hội Tụ Các Chỉ Số

1. **Động lực sụt giảm của Loss:** Trong Epoch 1, Train Loss khởi điểm ở mức 2.6980 và sụt giảm liên tục qua từng mẻ dữ liệu, đạt giá trị trung bình 2.6504. Validation Loss đạt mức xuất sắc 2.2326. Việc Validation Loss thấp hơn Train Loss ở giai đoạn đầu là hiện tượng kinh điển do tập Train phải chịu tải phép tăng cường hình học ngẫu nhiên D_4 liên tục, trong khi tập Validation được đánh giá trên các trạng thái tĩnh chuẩn mực.

2. **Sự bứt phá của Top-1 và Top-3 Policy Accuracy:** Ngay từ Epoch 1, mạng đã đạt **Top-1 Accuracy 44.41%** và **Top-3 Accuracy 70.62%** trên tập 36,077 mẫu kiểm định chưa từng thấy. Điều này chứng minh rằng thân mạng ResNet-6 với 64 kênh đặc trưng đã học được các mẫu hình học cơ bản của cờ Caro (các đường thẳng 3 quân, 4 quân, điểm giao tranh trung tâm) với độ tin cậy vượt trội.

3. **Hội tụ của Value Head MSE:** Hàm mất mát giá trị Value MSE khởi đầu ở mức 0.9573 và ổn định dần, chứng minh mạng đang xây dựng khả năng phán đoán thắng/thua song hành với việc gợi ý nước đi chiến thuật.

5.3 Kết Quả Đánh Giá Bộ Kiểm Chứng Chiến Thuật (Tactical Verification Suite)

Sau khi hoàn tất quá trình huấn luyện, mô hình được kiểm tra nghiêm ngặt trên 3 tình huống sát cục thực tế để kiểm chứng trí tuệ nhân tạo:

- **Kịch bản 1: Sát cục thắng ngay (Open-4 Immediate Win):** Khi quân ta có chuỗi 4 quân liên tiếp tại $(7,3),(7,4),(7,5),(7,6)$, mô hình dự đoán nước đi chiến thắng tại ô $(7,2)$ hoặc $(7,7)$ với xác suất áp đảo ($> 85\%$), đồng thời xuất ra điểm số kỳ vọng chiến thắng $V \approx +0.95$.
- **Kịch bản 2: Phòng ngự khẩn cấp (Urgent Defense Block):** Khi đối phương nắm chuỗi 4 quân nguy hiểm, mạng nơ-ron nhận diện chính xác các điểm chặn sinh tử tại hai đầu, đồng thời điểm số kỳ vọng tụt xuống mức đe dọa $V \approx -0.85$.
- **Kịch bản 3: Khai cuộc mở cờ (Center Opening Move):** Trên bàn cờ trống, xác suất nước đi tập trung cao nhất vào đúng ô trung tâm $(7,7)$ và các ô lân cận bán kính Chebyshev $r \leq 1$, hoàn toàn phù hợp với lý thuyết khai cuộc cờ Caro hiện đại.

Chapter 6

Kiến Trúc Xuất Khẩu ONNX & Tích Hợp Vào Engine Thực Tế

6.1 Chuẩn Mở ONNX (Open Neural Network Exchange)

Sau khi hoàn tất quá trình huấn luyện trên PyTorch, mô hình được xuất sang tệp:

$$\mathcal{F}_{\text{onnx}} = \texttt{caro_resnet.onnx} \tag{6.1}$$

với chuẩn giao thức **ONNX Opset 14**.

Lợi ích của việc đóng gói ONNX:

1. **Tách rời môi trường Python:** Engine trò chơi viết bằng Rust, C++ hoặc WebAssembly không cần cài đặt Python, CUDA Toolkit hay PyTorch cồng kềnh (hàng chục Gigabytes).
2. **Tối ưu hóa đồ thị tính toán tĩnh (Graph Optimization):** ONNX Runtime tự động thực hiện phép gộp tầng (*Layer Fusion*, ví dụ gộp Convolution + BatchNorm + ReLU thành 1 hạt nhân tính toán duy nhất), giảm thiểu số lần truy xuất bộ nhớ đệm và tăng tốc suy luận thêm 30–40%.
3. **Trục kích thước động (Dynamic Axes):** Khai báo `board_state` với trục mẻ linh hoạt cho phép mô hình vừa có thể suy luận đơn lẻ từng nước đi ($B = 1$), vừa có thể suy luận theo mẻ song song ($B = 16$ hoặc 64) khi phục vụ thuật toán tìm kiếm cây MCTS.

6.2 Đo Đạc Độ Trễ Suy Luận (Inference Latency Benchmark)

Với độ trễ suy luận chỉ ≈ 14.2 ms trên môi trường WebAssembly của trình duyệt, mô hình hoàn toàn đáp ứng ngân sách phản hồi thời gian thực (< 50 ms), sẵn sàng thay thế hoàn toàn hoặc đồng hành cùng thuật toán Heuristic hiện tại.

Table 6.1: Đo đạc độ trễ suy luận của mô hình caro_resnet.onnx trên các môi trường phần cứng khác nhau.

Môi trường thực thi	**Động cơ suy luận (Backend)**	**Độ trễ Batch 1**	**Độ trễ Batch 16**
NVIDIA T1200 Laptop GPU	TensorRT / CUDA Execution Provider	**2.1 ms**	**4.8 ms**
Intel Core i7 CPU (Host)	ONNX Runtime CPU (AVX2 / OpenMP)	**11.5 ms**	**28.2 ms**
Trình duyệt Web (WebAssembly)	ONNX Runtime Web / Rust `tract` crate	**14.2 ms**	**45.0 ms**

Chapter 7

Kết Luận: Chuyển Giao Sang Phase 3 (AlphaZero MCTS Self-Play)

7.1 Tổng Kết Đóng Góp Của Phase 2

Phase 2 đã hoàn thành trọn vẹn và xuất sắc việc hiện thực hóa mô hình nơ-ron:

1. Đã làm chủ toàn bộ nền tảng toán học ma trận và kiến trúc song song phần cứng CUDA trên GPU NVIDIA T1200.
2. Kiểm chứng tính đúng đắn và hiệu năng của kỹ thuật Automatic Mixed Precision (AMP FP16) kết hợp `GradScaler`.
3. Mạng nơ-ron hai đầu ResNet-6 đạt Top-1 Accuracy vượt mốc 45% và Top-3 Accuracy vượt mốc 70% chỉ sau các epoch đầu tiên.
4. Bộ kiểm chứng chiến thuật xác nhận mạng đã nắm bắt sâu sắc các quy tắc sát cục cơ bản (nước 4 thắng ngay, nước 4 cần chặn).
5. Xuất khẩu mô hình thành công sang chuẩn mở ONNX, sẵn sàng triển khai cross-platform.

7.2 Kế Hoạch Cho Phase 3: Tự Chơi Tăng Cường (AlphaZero MCTS)

Với mô hình `caro_resnet.onnx` đã được nung nấu vững chắc từ 10,000 ván cờ chuyên gia, Phase 3 sẽ mở ra cánh cổng tối thượng:

- Sử dụng mạng ResNet làm hàm phân phối tiên nghiệm (Prior Policy $P(s, a)$) và hàm lượng giá thế trận (Leaf Evaluation $V(s)$) cho thuật toán MCTS.
- Cho hai phiên bản mạng nơ-ron tự đối kháng với nhau hàng chục ngàn ván đấu.
- Sử dụng kết quả tìm kiếm MCTS làm mục tiêu cải tiến chính sách:

$$\pi_{\text{MCTS}}(a \mid s) = \frac{N(s, a)^{1/\tau}}{\sum_b N(s, b)^{1/\tau}} \tag{7.1}$$

giúp mạng vượt qua giới hạn của dữ liệu Heuristic ban đầu và đạt tới trình độ Đại Kiện Tướng cờ Caro.

Bibliography

[1] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., ... & Hassabis, D. (2017). *Mastering the game of Go without human knowledge*. Nature, 550(7676), 354-359.

[2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).

[3] Ioffe, S., & Szegedy, C. (2015). *Batch normalization: Accelerating deep network training by reducing internal covariate shift*. In International conference on machine learning (pp. 448-456). pmlr.

[4] Loshchilov, I., & Hutter, F. (2017). *Decoupled weight decay regularization*. arXiv preprint arXiv:1711.05101. (AdamW paper).

[5] Loshchilov, I., & Hutter, F. (2016). *SGDR: Stochastic gradient descent with warm restarts*. arXiv preprint arXiv:1608.03983. (Cosine Annealing LR).

[6] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., ... & Wu, H. (2017). *Mixed precision training*. arXiv preprint arXiv:1710.03740.

[7] NVIDIA Corporation. (2018). *NVIDIA Turing GPU Architecture Whitepaper: Next Generation Graphics and Deep Learning Acceleration*. Official NVIDIA Technical Documentation.